

ЛАБОРАТОРНАЯ РАБОТА № 8
«Введение в работу с XML, JSON, HTML и Excel»

Студент(ка): Зырянова Татьяна Евгеньевна

Группа: РИМ – 150950

Предмет: Программирование на Java

Репозиторий ДЗ: https://github.com/ZyrT12/java-core_2026-2027

ЦЕЛЬ РАБОТЫ

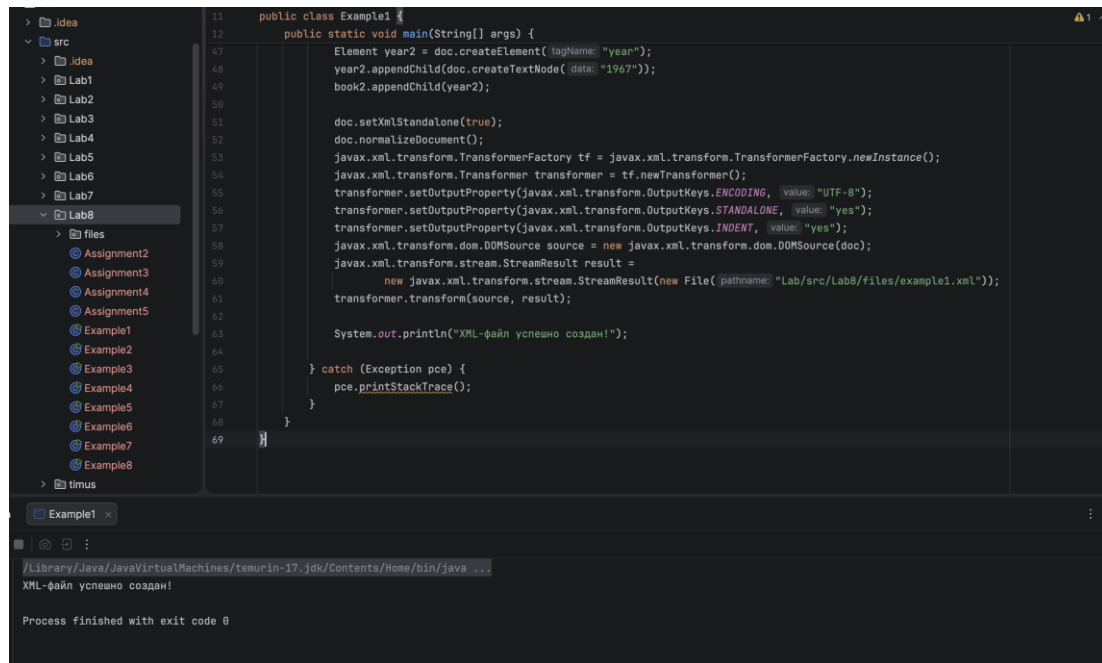
Получить представление о механизме многопоточности в языке программирования Java.

ОПИСАНИЕ ЗАДАЧИ

В данной лабораторной работе необходимо изучить работу с XML, JSON, HTML и Excel в Java. В ходе работы требуется выполнить примеры из раздела 1, реализовать доработки XML-, JSON-, HTML- и Excel-парсеров, подготовить собственные файлы данных по варианту 4 «Список дней рождений людей (Birthday)», а также решить две задачи с сайта Timus.

ХОД ВЫПОЛНЕНИЯ РАБОТЫ

1) Создание XML-файла.



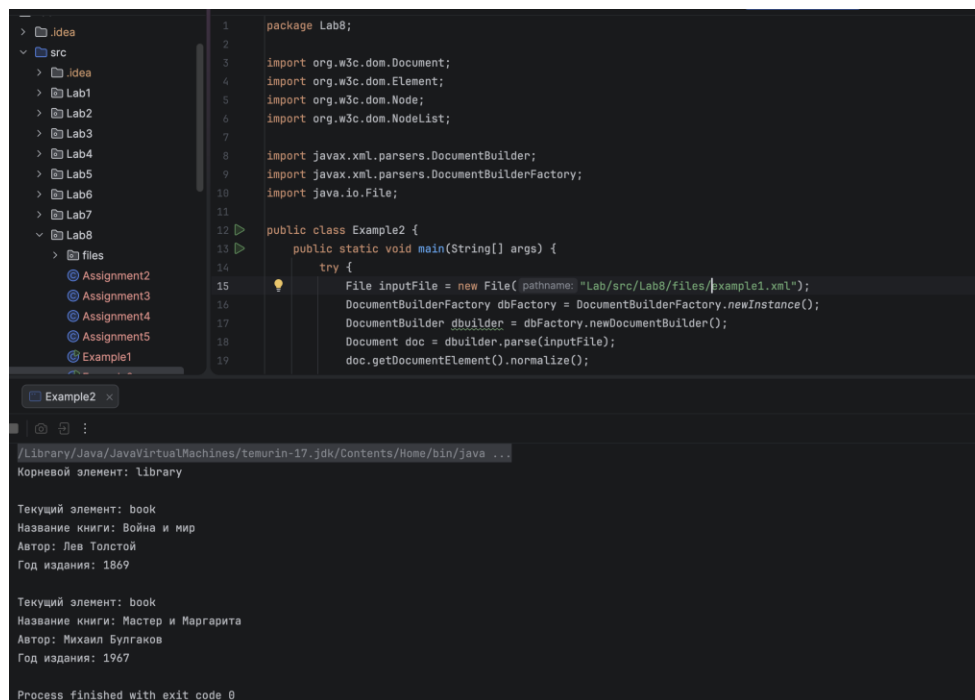
The screenshot shows an IDE with a project structure on the left. The 'src' directory contains a 'Lab8' folder, which in turn contains a 'files' folder. Inside 'files', there are several files including 'Example1'. The main editor displays the code for 'Example1.java'. The code uses the javax.xml.transform package to create an XML document. It sets the document to be standalone, normalizes it, and then uses a TransformerFactory to create a transformer. The transformer is configured with UTF-8 encoding, standalone mode, and indentation. It then transforms a DOMSource (the document) into a StreamResult, which is written to a file named 'example1.xml' in the 'Lab/src/Lab8/files' directory. The program prints a success message and catches any exceptions.

```
11 public class Example1 {  
12     public static void main(String[] args) {  
47         Element year2 = doc.createElement("year");  
48         year2.appendChild(doc.createTextNode("1967"));  
49         book2.appendChild(year2);  
50  
51         doc.setXmlStandalone(true);  
52         doc.normalizeDocument();  
53         javax.xml.transform.TransformerFactory tf = javax.xml.transform.TransformerFactory.newInstance();  
54         javax.xml.transform.Transformer transformer = tf.newTransformer();  
55         transformer.setOutputProperty(javax.xml.transform.OutputKeys.ENCODING, "UTF-8");  
56         transformer.setOutputProperty(javax.xml.transform.OutputKeys.STANDALONE, "yes");  
57         transformer.setOutputProperty(javax.xml.transform.OutputKeys.INDENT, "yes");  
58         javax.xml.transform.dom.DOMSource source = new javax.xml.transform.dom.DOMSource(doc);  
59         javax.xml.transform.stream.StreamResult result =  
60             new javax.xml.transform.stream.StreamResult(new File("Lab/src/Lab8/files/example1.xml"));  
61         transformer.transform(source, result);  
62  
63         System.out.println("XML-файл успешно создан!");  
64  
65     } catch (Exception pce) {  
66         pce.printStackTrace();  
67     }  
68 }  
69 }
```

The output window shows the command: `/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java ...` and the message: `XML-файл успешно создан!`. The process finished with exit code 0.

Рисунок 1. Результат работы программы Example1

2) Чтение XML-файла.



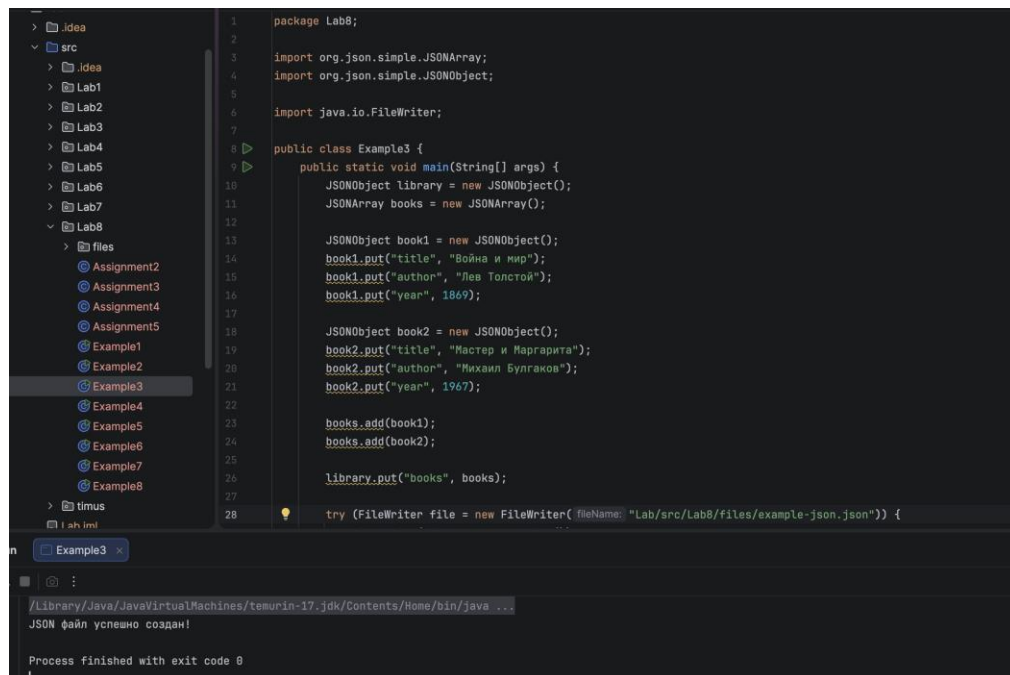
The screenshot shows the same IDE project structure. The 'Example2.java' file is now open in the editor. This program uses the javax.xml.parsers package to read the XML file 'example1.xml' from the 'Lab/src/Lab8/files' directory. It uses a DocumentBuilderFactory to create a DocumentBuilder, which then parses the XML file. The resulting Document object is used to retrieve the root element ('library') and the current element ('book'). The program then prints out the details of the book, including its title, author, and year of publication.

```
1 package Lab8;  
2  
3 import org.w3c.dom.Document;  
4 import org.w3c.dom.Element;  
5 import org.w3c.dom.Node;  
6 import org.w3c.dom.NodeList;  
7  
8 import javax.xml.parsers.DocumentBuilder;  
9 import javax.xml.parsers.DocumentBuilderFactory;  
10 import java.io.File;  
11  
12 public class Example2 {  
13     public static void main(String[] args) {  
14         try {  
15             File inputFile = new File("Lab/src/Lab8/files/example1.xml");  
16             DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();  
17             DocumentBuilder dbuilder = dbFactory.newDocumentBuilder();  
18             Document doc = dbuilder.parse(inputFile);  
19             doc.getDocumentElement().normalize();  
20  
21             // Output: Корневой элемент: library  
22             // Текущий элемент: book  
23             // Название книги: Война и мир  
24             // Автор: Лев Толстой  
25             // Год издания: 1869  
26  
27             // Текущий элемент: book  
28             // Название книги: Мастер и Маргарита  
29             // Автор: Михаил Булгаков  
30             // Год издания: 1967  
31  
32         } catch (Exception e) {  
33             e.printStackTrace();  
34         }  
35     }  
36 }
```

The output window shows the command: `/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java ...` and the message: `Корневой элемент: library`. The process finished with exit code 0.

Рисунок 2. Результат работы программы Example2

3) Создание JSON-файла.



```
1 package Lab8;
2
3 import org.json.simple.JSONArray;
4 import org.json.simple.JSONObject;
5
6 import java.io.FileWriter;
7
8 public class Example3 {
9     public static void main(String[] args) {
10         JSONObject library = new JSONObject();
11         JSONArray books = new JSONArray();
12
13         JSONObject book1 = new JSONObject();
14         book1.put("title", "Война и мир");
15         book1.put("author", "Лев Толстой");
16         book1.put("year", 1869);
17
18         JSONObject book2 = new JSONObject();
19         book2.put("title", "Мастер и Маргарита");
20         book2.put("author", "Михаил Булгаков");
21         book2.put("year", 1967);
22
23         books.add(book1);
24         books.add(book2);
25
26         library.put("books", books);
27
28         try (FileWriter file = new FileWriter("Lab/src/Lab8/files/example-json.json")) {
```

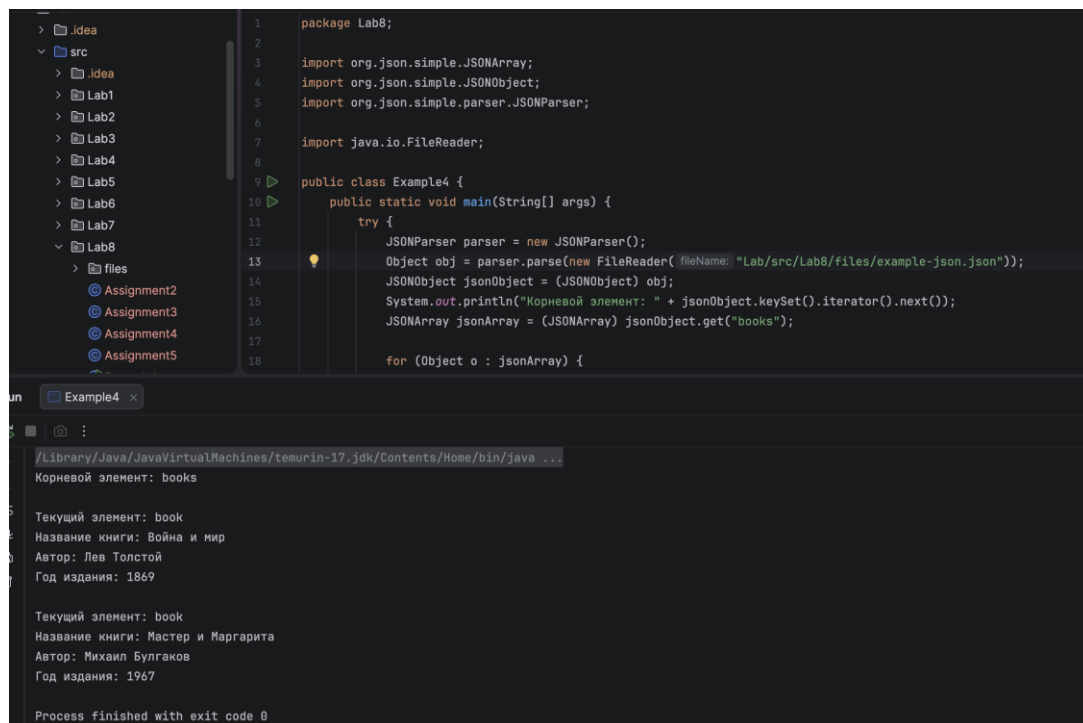
Example3

/Library/Java/JavaVirtualMachines/temurin-17-jdk/Contents/Home/bin/java ...
JSON файл успешно создан!

Process finished with exit code 0

Рисунок 3. Результат работы программы Example3

4) Чтение JSON-файла.



```
1 package Lab8;
2
3 import org.json.simple.JSONArray;
4 import org.json.simple.JSONObject;
5 import org.json.simple.parser.JSONParser;
6
7 import java.io.FileReader;
8
9 public class Example4 {
10     public static void main(String[] args) {
11         try {
12             JSONParser parser = new JSONParser();
13             Object obj = parser.parse(new FileReader("Lab/src/Lab8/files/example-json.json"));
14             JSONObject jsonObject = (JSONObject) obj;
15             System.out.println("Корневой элемент: " + jsonObject.keySet().iterator().next());
16             JSONArray jsonArray = (JSONArray) jsonObject.get("books");
17
18             for (Object o : jsonArray) {
```

Example4

/Library/Java/JavaVirtualMachines/temurin-17-jdk/Contents/Home/bin/java ...
Корневой элемент: books

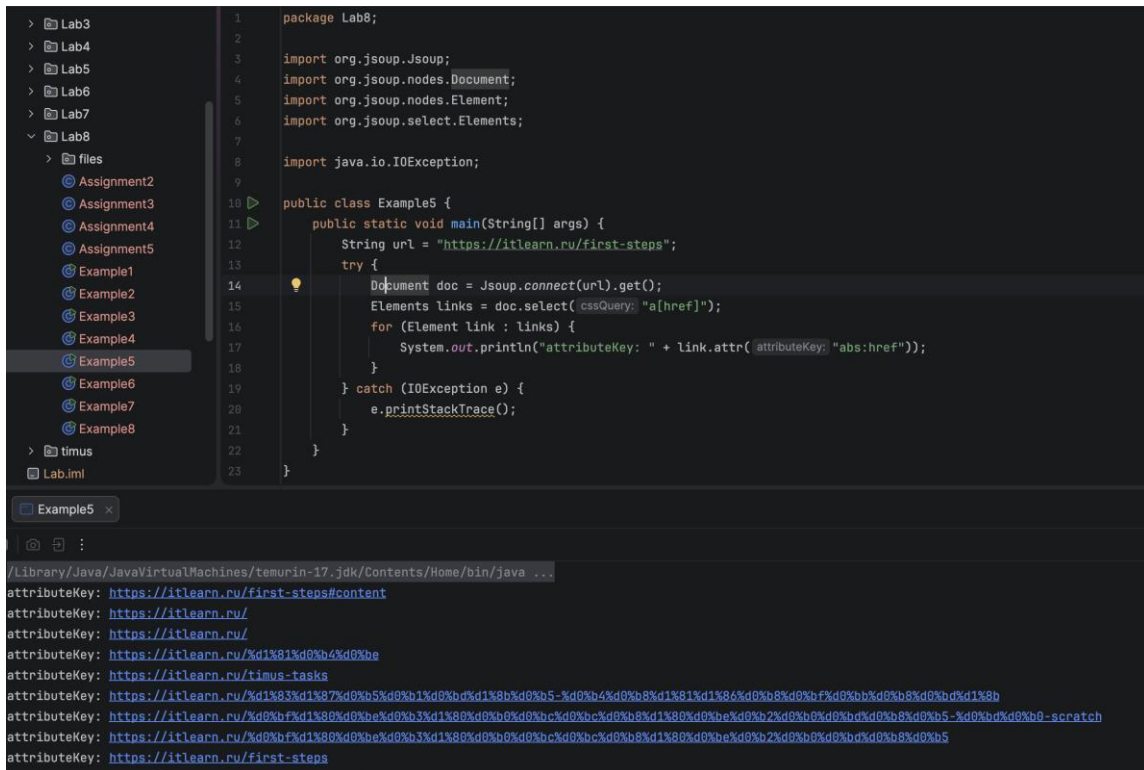
Текущий элемент: book
Название книги: Война и мир
Автор: Лев Толстой
Год издания: 1869

Текущий элемент: book
Название книги: Мастер и Маргарита
Автор: Михаил Булгаков
Год издания: 1967

Process finished with exit code 0

Рисунок 4. Результат работы программы Example4

5) Парсинг HTML-страницы со ссылками.



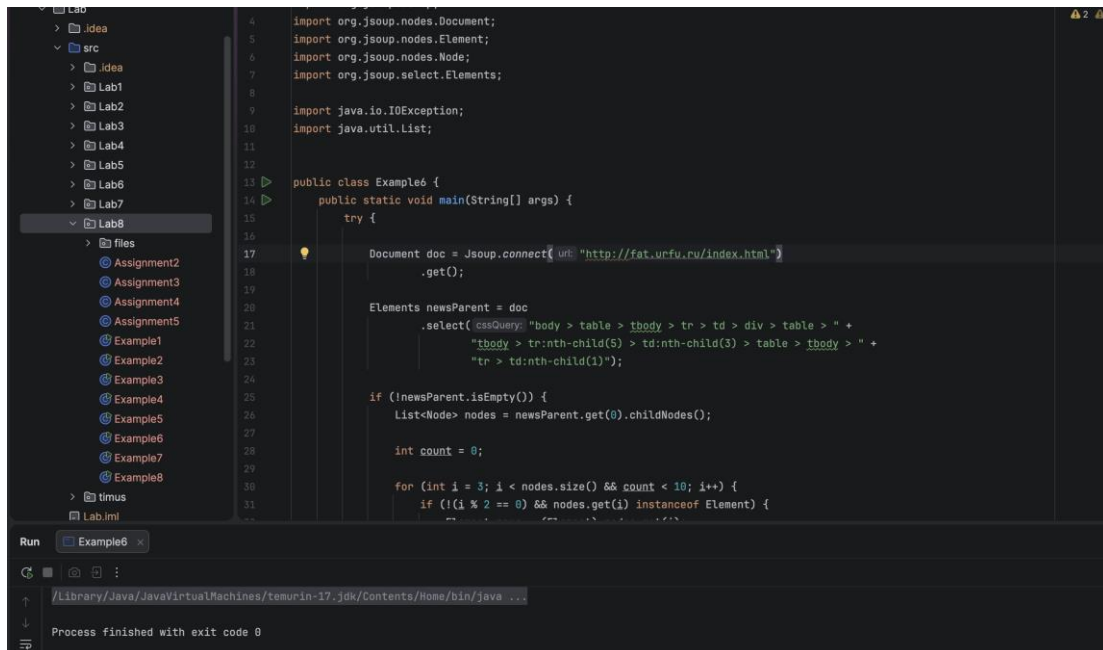
```
1 package Lab8;
2
3 import org.jsoup.Jsoup;
4 import org.jsoup.nodes.Document;
5 import org.jsoup.nodes.Element;
6 import org.jsoup.select.Elements;
7
8 import java.io.IOException;
9
10 public class Example5 {
11     public static void main(String[] args) {
12         String url = "https://itlearn.ru/first-steps";
13         try {
14             Document doc = Jsoup.connect(url).get();
15             Elements links = doc.select(cssQuery: "a[href]");
16             for (Element link : links) {
17                 System.out.println("attributeKey: " + link.attr(attributeKey: "abs:href"));
18             }
19         } catch (IOException e) {
20             e.printStackTrace();
21         }
22     }
23 }
```

Output:

```
attributeKey: https://itlearn.ru/first-steps#content
attributeKey: https://itlearn.ru/
attributeKey: https://itlearn.ru/
attributeKey: https://itlearn.ru/%d1%81%d0%b4%d0%be
attributeKey: https://itlearn.ru/timur-tasks
attributeKey: https://itlearn.ru/%d1%83%d1%87%d0%b5%d0%b1%d0%bd%d1%8b%d0%b5-%d0%b4%d0%b8%d1%81%d1%86%d0%b8%d0%bf%d0%bb%d0%b8%d0%bd%d1%8b
attributeKey: https://itlearn.ru/%d0%bf%d1%80%d0%be%d0%b3%d1%88%d0%b0%d0%bc%d0%bc%d0%b8%d1%80%d0%be%d0%b2%d0%b0%d0%bd%d0%b8%d0%b5-%d0%bd%d0%b8-scratch
attributeKey: https://itlearn.ru/%d0%bf%d1%80%d0%be%d0%b3%d1%88%d0%b0%d0%bc%d0%bc%d0%b8%d1%80%d0%be%d0%b2%d0%b8%d0%bd%d0%b8%d0%b5
attributeKey: https://itlearn.ru/first-steps
```

Рисунок 5. Результат работы программы Example5

6) Парсинг HTML-страницы с новостями.



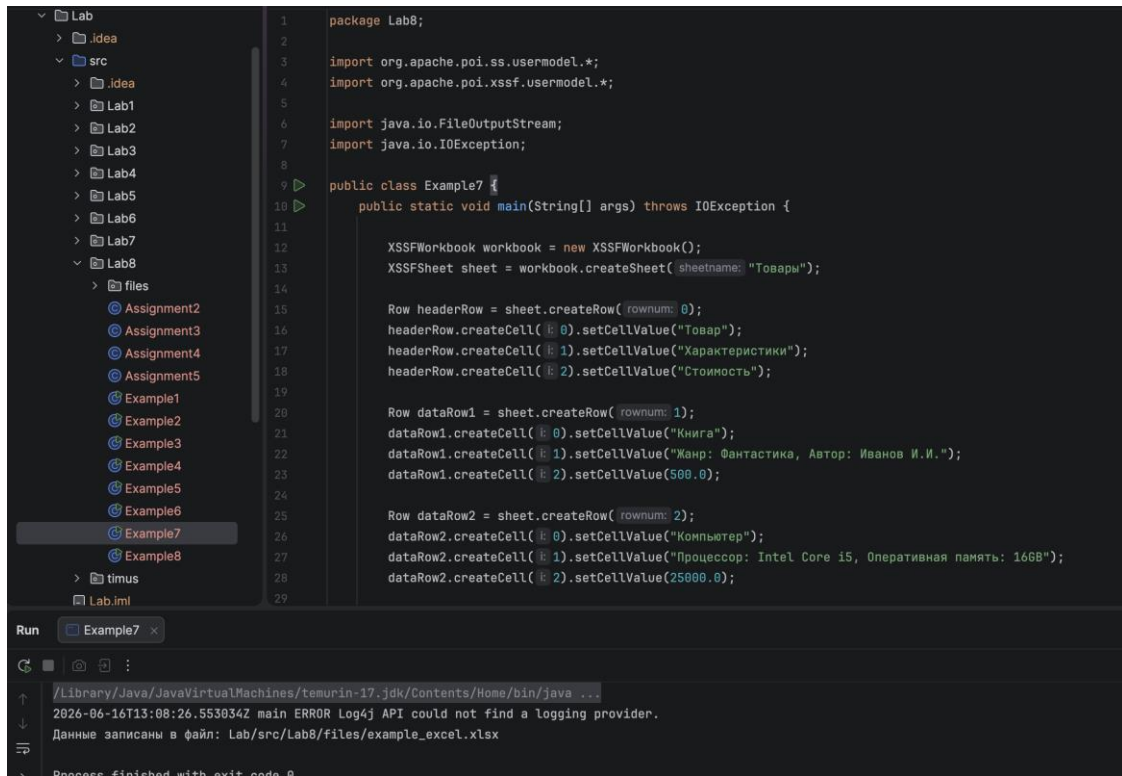
```
4 import org.jsoup.nodes.Document;
5 import org.jsoup.nodes.Element;
6 import org.jsoup.nodes.Node;
7 import org.jsoup.select.Elements;
8
9 import java.io.IOException;
10 import java.util.List;
11
12
13 public class Example6 {
14     public static void main(String[] args) {
15         try {
16             Document doc = Jsoup.connect(url: "http://fat.wrfu.ru/index.html")
17                 .get();
18
19             Elements newsParent = doc
20                 .select(cssQuery: "body > table > tbody > tr > td > div > table > " +
21                     "tbody > tr:nth-child(5) > td:nth-child(3) > table > tbody > " +
22                     "tr > td:nth-child(1)");
23
24             if (!newsParent.isEmpty()) {
25                 List<Node> nodes = newsParent.get(0).childNodes();
26
27                 int count = 0;
28
29                 for (int i = 3; i < nodes.size() && count < 10; i++) {
30                     if ((i % 2 == 0) && nodes.get(i) instanceof Element) {
31                         // ...
32                     }
33                 }
34             }
35         } catch (IOException e) {
36             e.printStackTrace();
37         }
38     }
39 }
```

Run Example6

Process finished with exit code 0

Рисунок 6. Результат работы программы Example6

7) Создание Excel-файла.



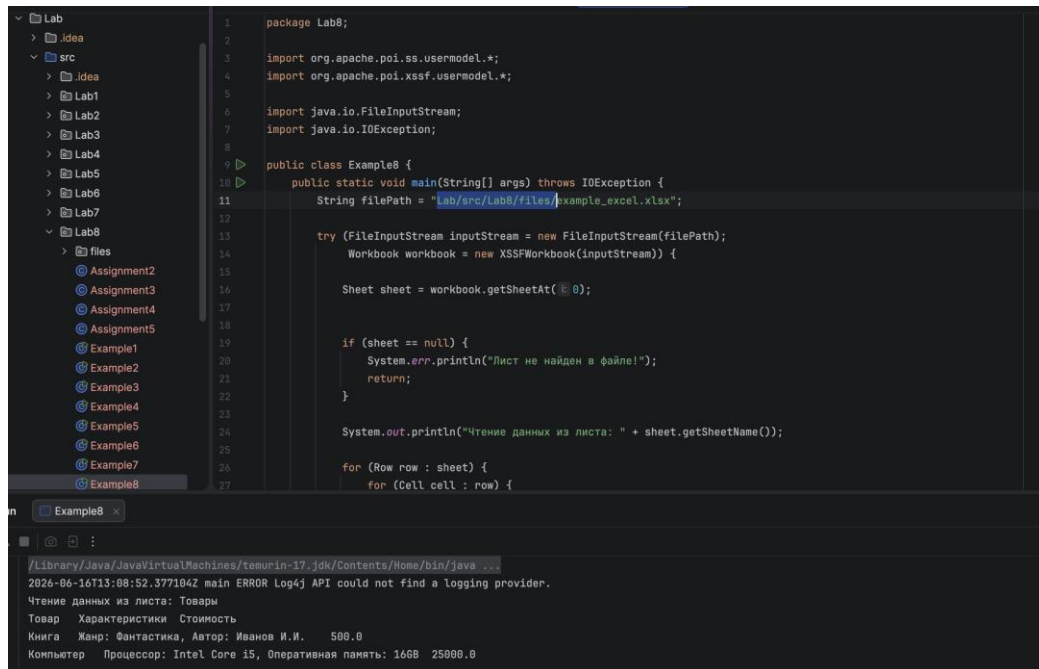
```
1 package Lab8;
2
3 import org.apache.poi.ss.usermodel.*;
4 import org.apache.poi.xssf.usermodel.*;
5
6 import java.io.FileOutputStream;
7 import java.io.IOException;
8
9 public class Example7 {
10     public static void main(String[] args) throws IOException {
11
12         XSSFWorkbook workbook = new XSSFWorkbook();
13         XSSFSheet sheet = workbook.createSheet("Товары");
14
15         Row headerRow = sheet.createRow(0);
16         headerRow.createCell(0).setCellValue("Товар");
17         headerRow.createCell(1).setCellValue("Характеристики");
18         headerRow.createCell(2).setCellValue("Стоимость");
19
20         Row dataRow1 = sheet.createRow(1);
21         dataRow1.createCell(0).setCellValue("Книга");
22         dataRow1.createCell(1).setCellValue("Жанр: Фантастика, Автор: Иванов И.И.");
23         dataRow1.createCell(2).setCellValue(500.0);
24
25         Row dataRow2 = sheet.createRow(2);
26         dataRow2.createCell(0).setCellValue("Компьютер");
27         dataRow2.createCell(1).setCellValue("Процессор: Intel Core i5, Оперативная память: 16GB");
28         dataRow2.createCell(2).setCellValue(25000.0);
29     }
30 }
```

Run Example7

2026-06-16T13:08:26.553034Z main ERROR Log4j API could not find a logging provider.
Данные записаны в файл: Lab/src/Lab8/files/example_excel.xlsx
Process finished with exit code 0

Рисунок 7. Результат работы программы Example7

8) Чтение Excel-файла.



```
1 package Lab8;
2
3 import org.apache.poi.ss.usermodel.*;
4 import org.apache.poi.xssf.usermodel.*;
5
6 import java.io.FileInputStream;
7 import java.io.IOException;
8
9 public class Example8 {
10     public static void main(String[] args) throws IOException {
11         String filePath = "Lab/src/Lab8/files/example_excel.xlsx";
12
13         try (FileInputStream inputStream = new FileInputStream(filePath);
14             Workbook workbook = new XSSFWorkbook(inputStream)) {
15
16             Sheet sheet = workbook.getSheetAt(0);
17
18             if (sheet == null) {
19                 System.err.println("Лист не найден в файле!");
20                 return;
21             }
22
23             System.out.println("Чтение данных из листа: " + sheet.getSheetName());
24
25             for (Row row : sheet) {
26                 for (Cell cell : row) {
27                     System.out.print(cell.getStringCellValue() + " ");
28                 }
29                 System.out.println();
30             }
31         }
32     }
33 }
```

Run Example8

2026-06-16T13:08:52.377104Z main ERROR Log4j API could not find a logging provider.
Чтение данных из листа: Товары
Товар Характеристики Стоимость
Книга Жанр: Фантастика, Автор: Иванов И.И. 500.0
Компьютер Процессор: Intel Core i5, Оперативная память: 16GB 25000.0

Рисунок 8. Результат работы программы Example8

ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

1. Выполнить все примеры из раздела 1.
2. Доработка XML-парсера по варианту Birthday.
3. Доработка JSON-парсера по варианту Birthday.
4. Доработка HTML-парсера.
5. Доработка Excel-парсера.
6. Решите две любые задачи с сайта Timus.

2) Доработка XML-парсера по варианту Birthday.

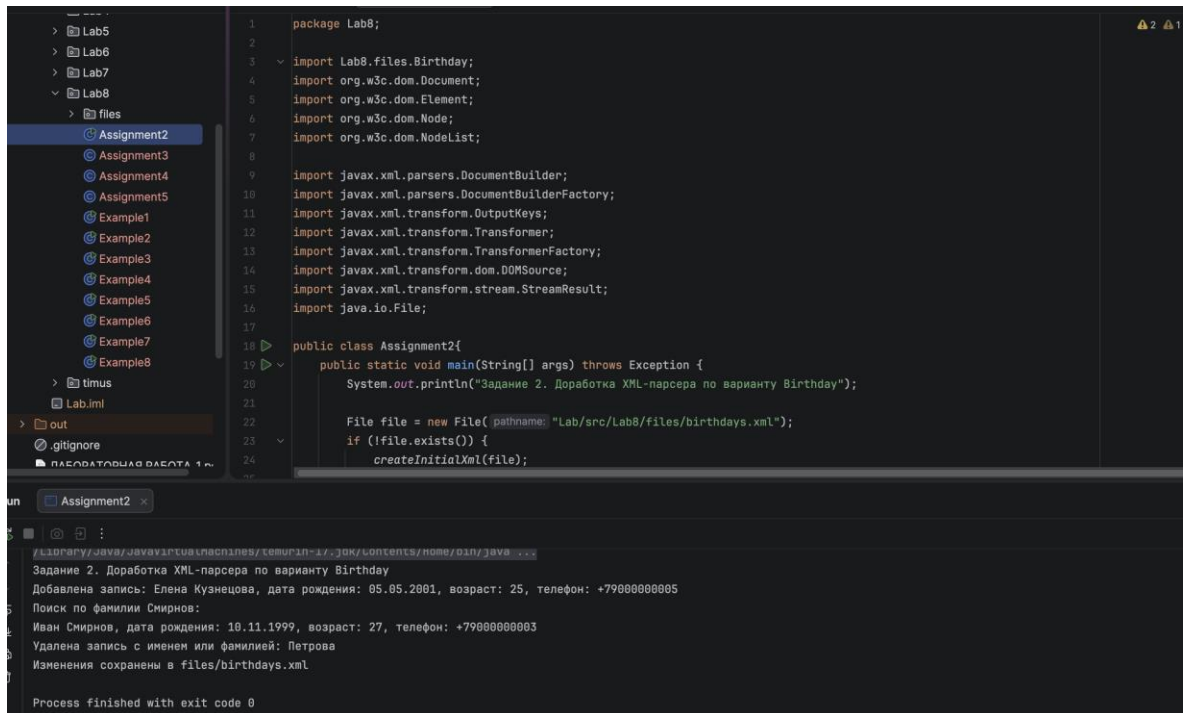


Рисунок 9. Результат работы программы XmlBirthdayParser

3) Доработка JSON-парсера по варианту Birthday.

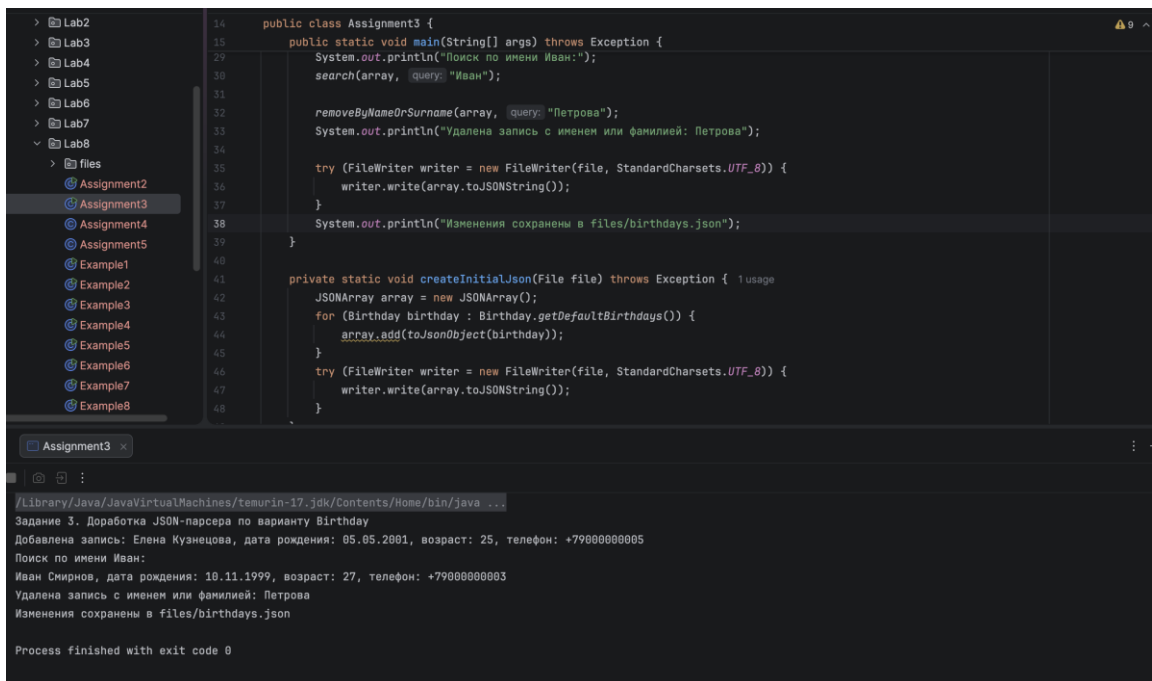
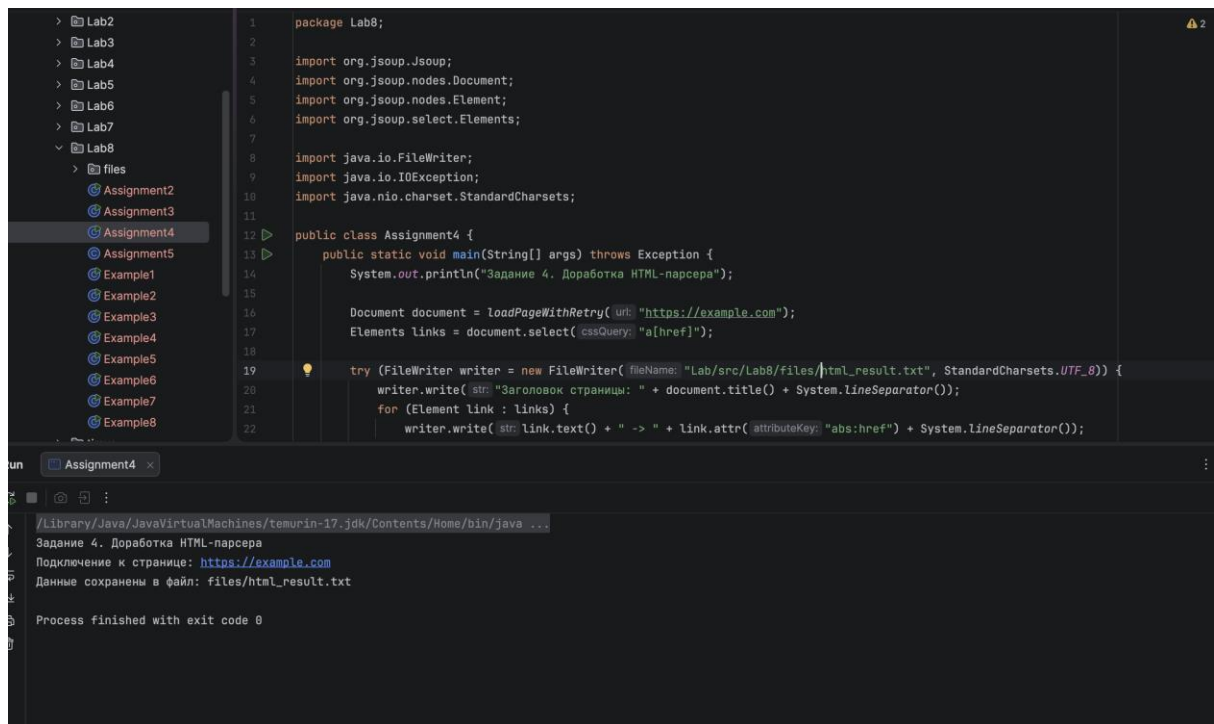


Рисунок 10. Результат работы программы JsonBirthdayParser

4) Доработка HTML-парсера.



```
1 package Lab8;
2
3 import org.jsoup.Jsoup;
4 import org.jsoup.nodes.Document;
5 import org.jsoup.nodes.Element;
6 import org.jsoup.select.Elements;
7
8 import java.io.FileWriter;
9 import java.io.IOException;
10 import java.nio.charset.StandardCharsets;
11
12 public class Assignment4 {
13     public static void main(String[] args) throws Exception {
14         System.out.println("Задание 4. Доработка HTML-парсера");
15
16         Document document = loadPageWithRetry( url: "https://example.com");
17         Elements links = document.select( cssQuery: "a[href]");
18
19         try (FileWriter writer = new FileWriter( fileName: "Lab/src/Lab8/files/html_result.txt", StandardCharsets.UTF_8)) {
20             writer.write( str: "Заголовок страницы: " + document.title() + System.LineSeparator());
21             for (Element link : links) {
22                 writer.write( str: link.text() + " -> " + link.attr( attributeKey: "abs:href") + System.LineSeparator());
23             }
24         }
```

run Assignment4 x

/Library/Java/JavaVirtualMachines/temurin-17-jdk/Contents/Home/bin/java ...

Задание 4. Доработка HTML-парсера

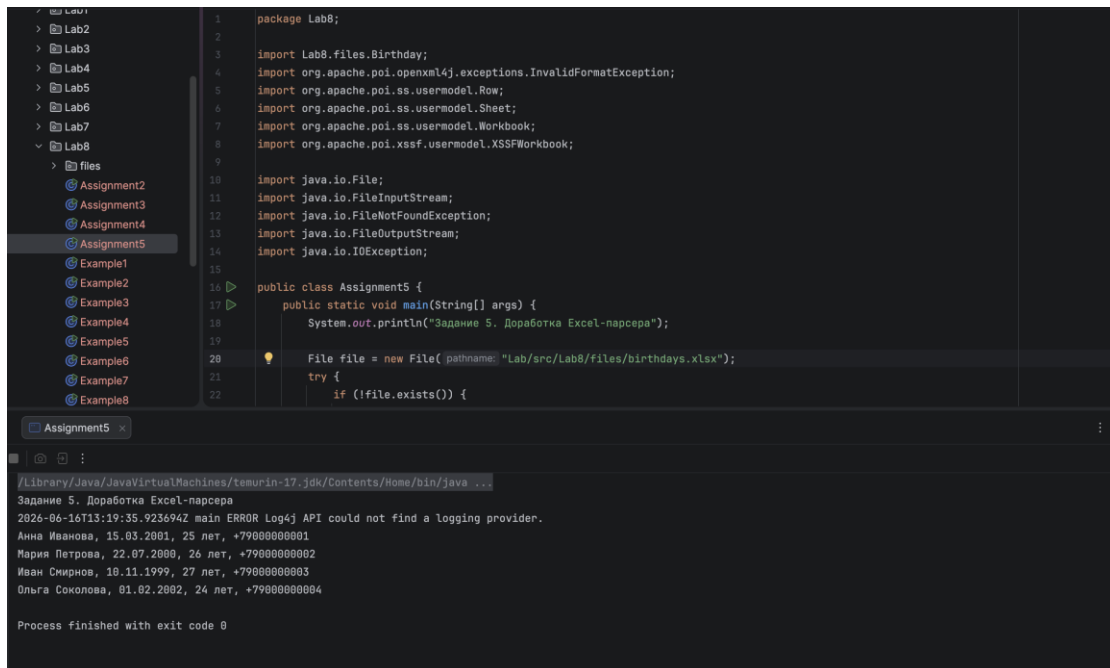
Подключение к странице: <https://example.com>

Данные сохранены в файл: files/html_result.txt

Process finished with exit code 0

Рисунок 11. Результат работы программы HtmlParserImproved

5) Доработка Excel-парсера.



```
1 package Lab8;
2
3 import Lab8.files.Birthday;
4 import org.apache.poi.openxml4j.exceptions.InvalidFormatException;
5 import org.apache.poi.ss.usermodel.Row;
6 import org.apache.poi.ss.usermodel.Sheet;
7 import org.apache.poi.ss.usermodel.Workbook;
8 import org.apache.poi.xssf.usermodel.XSSFWorkbook;
9
10 import java.io.File;
11 import java.io.FileInputStream;
12 import java.io.FileNotFoundException;
13 import java.io.FileOutputStream;
14 import java.io.IOException;
15
16 public class Assignment5 {
17     public static void main(String[] args) {
18         System.out.println("Задание 5. Доработка Excel-парсера");
19
20         File file = new File( pathname: "Lab/src/Lab8/files/birthdays.xlsx");
21         try {
22             if (!file.exists()) {
23                 // ...
24             }
25         }
```

Assignment5 x

/Library/Java/JavaVirtualMachines/temurin-17-jdk/Contents/Home/bin/java ...

Задание 5. Доработка Excel-парсера

2026-06-16T13:19:35.923694Z main ERROR Log4j API could not find a logging provider.

Анна Иванова, 15.03.2001, 25 лет, +79000000001

Мария Петрова, 22.07.2000, 26 лет, +79000000002

Иван Смирнов, 10.11.1999, 27 лет, +79000000003

Ольга Соколова, 01.02.2002, 24 лет, +79000000004

Process finished with exit code 0

Рисунок 12. Результат работы программы ExcelBirthdayParser

РЕШЕНИЕ ЗАДАЧ TIMUS

1786. Биография Сандро

Ограничение времени: 1.0 секунды

Ограничение памяти: 64 МБ

Леогиус искал в библиотеке книгу, которую ему посоветовал учитель теоретической магии. Вдруг он наткнулся на древнюю летопись, записанную на нескольких листах пергамента. Бегло проглядев её, Леогиус понял, что она описывает жизнь и удивительные приключения одного лича. Неужели этот манускрипт — биография лича Сандро, утерянная много веков назад? Если это так, находку нужно как можно быстрее показать Верховному Совету Магов. Одна беда — в тексте летописи ни разу не упоминается имя Сандро. Что же делать? Совет может и не поверить, что летопись повествует именно о Сандро.

Леогиус решил, что летопись нужно срочно исправить. Довольно быстро нашёлся маг, готовый сделать это. Но за хорошую работу всегда нужно платить. Корректор был готов исправить любую букву на любую другую букву того же регистра (заглавную — на заглавную, строчную — на строчную) за пять золотых. Кроме того, он мог изменить регистр любой буквы, также за пять золотых. Помогите Леогиусу определить, какое минимальное количество золотых ему придётся заплатить, чтобы в тексте появилась строка «Sandro».

Исходные данные

Единственная строка содержит текст летописи, состоящий из строчных и заглавных латинских букв. В тексте не менее шести и не более 200 символов.

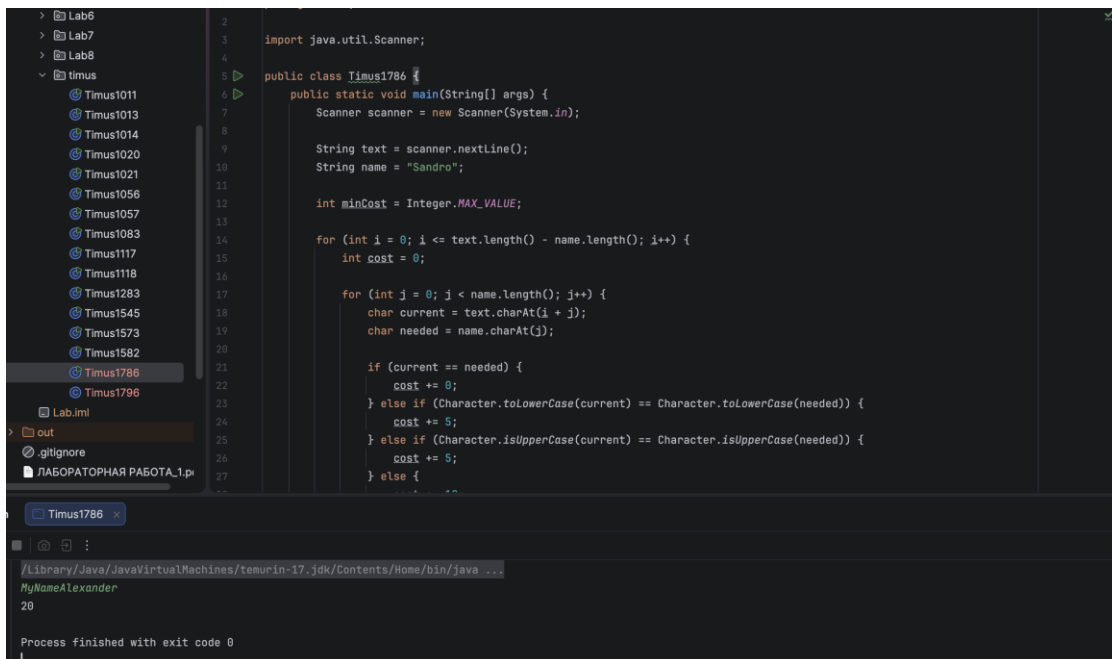
Результат

Выведите минимальное количество золотых, которое придётся заплатить, чтобы в тексте появилось имя Сандро.

Пример

исходные данные	результат
MyNameIsAlexander	20

Рисунок 13. Условие задачи Timus 1786



```
import java.util.Scanner;

public class Timus1786 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        String text = scanner.nextLine();
        String name = "Sandro";

        int minCost = Integer.MAX_VALUE;

        for (int i = 0; i <= text.length() - name.length(); i++) {
            int cost = 0;

            for (int j = 0; j < name.length(); j++) {
                char current = text.charAt(i + j);
                char needed = name.charAt(j);

                if (current == needed) {
                    cost += 0;
                } else if (Character.toLowerCase(current) == Character.toLowerCase(needed)) {
                    cost += 5;
                } else if (Character.toUpperCase(current) == Character.toUpperCase(needed)) {
                    cost += 5;
                } else {
                    cost += 10;
                }
            }

            minCost = Math.min(minCost, cost);
        }

        System.out.println(minCost);
    }
}
```

MyNameAlexander

20

Process finished with exit code 0

Рисунок 14. Решение задачи Timus 1786

1796. Парк аттракционов

Ограничение времени: 1.0 секунды

Ограничение памяти: 64 МБ

В солнечное воскресенье группа детей во главе с учителем отправилась в парк аттракционов. В тот день на кассе в парке сидела очень добрая и спокойная тётя Фрося. Учитель отдал ей деньги, но не сказал, сколько билетов ему нужно. Может ли тётя Фрося определить это только по набору купюр, которые дал ей учитель? Она считает, что учитель не давал лишних купюр, то есть, если убрать из набора любую купюру, то денег на все билеты уже не хватит.

Исходные данные

В первой строке через пробел записаны шесть неотрицательных целых чисел — количества купюр достоинством 10, 50, 100, 500, 1000 и 5000 рублей, соответственно, которыми учитель расплатился с тётей Фросей. Во второй строке записано целое положительное число — цена одного билета. Все числа во входных данных не превосходят 1000.

Результат

Определите, сколько билетов хотел купить учитель. В первой строке выведите количество вариантов ответа, а во второй — все эти варианты через пробел, упорядоченные по возрастанию. Гарантируется, что хотя бы один вариант ответа существует.

Примеры

исходные данные	результат
0 2 0 0 0 0 10	5 6 7 8 9 10
1 2 0 0 0 0 10	1 11

Рисунок 15. Условие задачи Timus 1796

```
package timus;

import java.util.Scanner;

public class Timus1796 {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int[] values = {10, 50, 100, 500, 1000, 5000};
        int[] count = new int[6];

        int totalMoney = 0;
        int minBanknote = Integer.MAX_VALUE;

        for (int i = 0; i < 6; i++) {
            count[i] = scanner.nextInt();

            if (count[i] > 0) {
                totalMoney += count[i] * values[i];

                if (values[i] < minBanknote) {
                    minBanknote = values[i];
                }
            }
        }

        int ticketPrice = scanner.nextInt();

        // Output logic would go here, based on the problem statement and examples.
```

Рисунок 16. Решение задачи Timus 1796

ВЫВОД

В ходе выполнения лабораторной работы были изучены способы работы с XML, JSON, HTML и Excel в языке Java. Были реализованы примеры создания и чтения XML- и JSON-файлов, парсинга HTML-страниц с помощью Jsoup, а также создания и чтения Excel-файлов с использованием Apache POI. В самостоятельной работе были выполнены доработки парсеров для варианта 4 «Список дней рождений людей (Birthday)»: реализованы добавление, поиск и удаление записей, сохранение результатов парсинга HTML в файл и обработка ошибок при чтении Excel-файлов.